

Combinatorics — Counting Principles, Permutations and Combinations

A 1.5-2 hour lecture on systematic counting, factorials, permutations, combinations and CS applications.

- Recommended teaching duration: approximately 1.5-2 hours including board work, worked examples, Python demonstrations and student discussion.
- Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

- Concept introduction and terminology
- Detailed definitions and notation
- Worked mathematical examples
- CS case studies with complete solutions
- Python implementation and verification
- Extended question bank
- 2-3 unsolved student practice questions at the end

1. Why Combinatorics?

Computer science often asks how many configurations, passwords, test suites, schedules, routes, or selections are possible. Counting methods avoid exhaustive listing when the search space is large.

2. Addition Principle

If alternatives are disjoint and can be completed in m and n ways, the total is $m+n$.

3. Multiplication Principle

If a process has m choices at one stage and n choices at the next, the total is mn . Extend this across multiple stages.

4. Factorial

$n! = n(n-1)\dots 1$ and $0! = 1$. Factorials count arrangements of n distinct objects.

5. Permutations

Order matters. $P(n,r) = n!/(n-r)!$. Example: selecting a president, vice president and secretary from a group is an ordered selection because roles differ.

6. Combinations

Order does not matter. $C(n,r) = n!/[r!(n-r)!]$. Selecting a team or a set of test cases is usually a combination.

7. Repetition

Always ask whether repetition is allowed. Passwords, codes and selections can have very different counts depending on this condition.

8. Case Study — Password Security

A simplified password has 3 distinct uppercase letters followed by 2 distinct digits, with no repetition.

Solution: $26P3 \times 10P2 = 26 \times 25 \times 24 \times 10 \times 9 = 1,404,000$ possible passwords.

9. Case Study — Software Testing

Choose 4 distinct test cases from 20 for a regression subset. Order does not matter.

Solution: $C(20,4) = 4,845$ subsets.

10. Case Study — Team Assignment

Select 3 developers from 10 and then appoint one selected developer as team lead.

Solution: $C(10,3) \times 3 = 120 \times 3 = 360$ outcomes.

11. Case Study — Menu Configuration

A software platform lets a user choose 1 of 4 themes and 1 of 3 layouts. How many configurations?

Solution: $4 \times 3 = 12$. If an optional language can also be selected from 5 choices, total becomes $4 \times 3 \times 5 = 60$.

12. Python Verification

`math.factorial`, `math.perm` and `math.comb` verify formulas. `itertools` can enumerate small examples and compare the number generated with the formula.

Python Laboratory

Python Activity 1

```
import math
print(math.factorial(5))
print(math.perm(26,3))
print(math.comb(20,4))
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
from itertools import permutations, combinations
items=['A','B','C','D']
print(len(list(permutations(items,2)))) # 12
print(len(list(combinations(items,2)))) # 6
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 3

```
# Password count
total = math.perm(26,3) * math.perm(10,2)
print(total) # 1404000
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5–2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Apply the addition principle to two disjoint choices.
2. Apply the multiplication principle to a multi-stage configuration.
3. Calculate a factorial.
4. Calculate $P(n,r)$.
5. Calculate $C(n,r)$.
6. Explain whether order matters in a given scenario.
7. Count passwords under a specified repetition policy.
8. Count possible software test subsets.
9. Count project teams with a designated leader.
10. Count interface configurations.
11. Verify a counting result with `math.perm` or `math.comb`.
12. Use `itertools` for a small enumeration and compare its length with the formula.
13. Explain how allowing repetition changes a password count.

End-of-Lecture Student Practice — No Solutions

1. Count a password space with and without repetition.
2. Count a test-team selection and explain why combination is used.
3. Verify a small counting case by enumeration in Python.

Quick Recap

Combinatorics counts possibilities systematically and supports security, testing and configuration analysis.